JVM overview

29th Study.

1. Virtual Machine
2. Types of Virtual Machine
   - Hardware based
   - Application based
3. Basic architecture of JVM
4. class loader Subsystem
   - loading
   - Linking
   - Initialization
5. Types of class loaders
   - Bootstrap class loader
   - Extension class loader
   - Application class loader..
6. How class loader works
7. What is the need of Customized class loader..
8. Psudo code for customized class loader..
9. Various Memory area of JVM...
   1. Method area
   2. Heap
   3. Stack
   4. PC registers..
   5. Native method stacks..
10. Program to display heap memory statistics..
11. How to get maximum and minimum heap size..
12. Execution engine
    1. Interpreter 2. JIT compiler..
13. Java Native Interface
14. complete architecture Diagram of JVM..
15. class file structure..

1.

Virtual Machine

It is a software simulation of a machine which can perform operations like a physical machines..

There are two types of virtual machines –

(i) Hardware based or System based VM

(ii) Application based or process based VM

(i) Hardware based / System based VM

It provides several logical systems on the same computer with strong isolation from each other.. that is on one physical machine we're defining multiple logical machines..

The main advantage of hardware based virtual machines is hardware resources sharing and improves utilization of hardware resources..

ex: KVM [Kernel-based virtual machine]
for Linux systems.

VM ware, Xen, cloud computing etc.

(ii) Application based or process based VM

These virtual machines access runtime engines to run a particular programming language applications..

ex: JVM – access a runtime ~~ag~~ engine to run Java based apps.

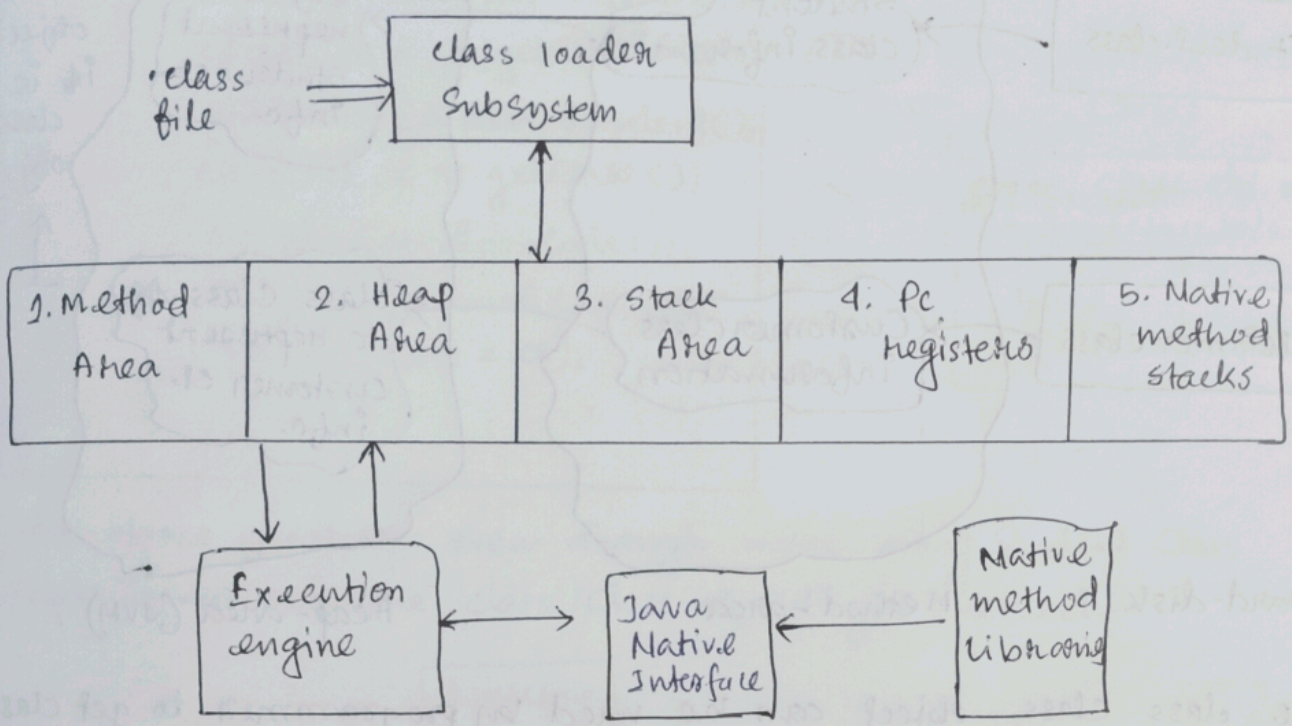
PVM – Perl
access runtime engine to run Perl based applications..

CLR – common language runtime
access runtime engine to run .net based applications..

JVM

JVM is the part of JRE and it is responsible to laad and run java class files..

Basic architecture diagram of JVM



class loader Sub-System :

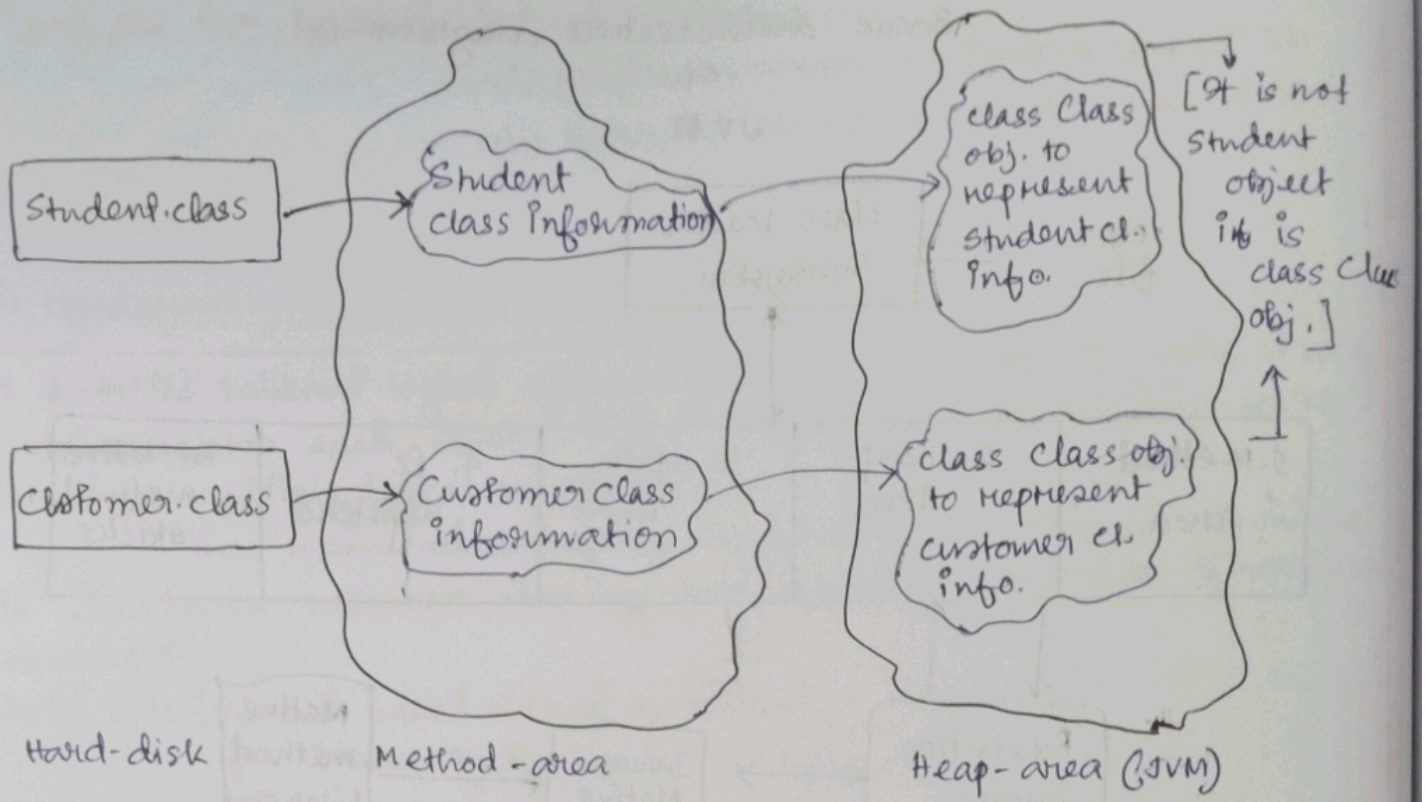
Class loader Sub-system is responsible for the following 3 activities (i) loading (ii) linking (iii) Initialization.

(i) loading :

loading means reading class file and store corresponding binary data in method area.. for each class file JVM will store corresponding information in method area.—

(a) fully qualified name of class.
(b) " " " " immediate parent class
(c) methods information...
(d) Variable ".
(e) Constructor "
(f) Modifiers information..
(g) constant-pool information etc...

After loading .class file immediately JVM creates an object for that loaded class on the heap memory of type java.lang.Class.



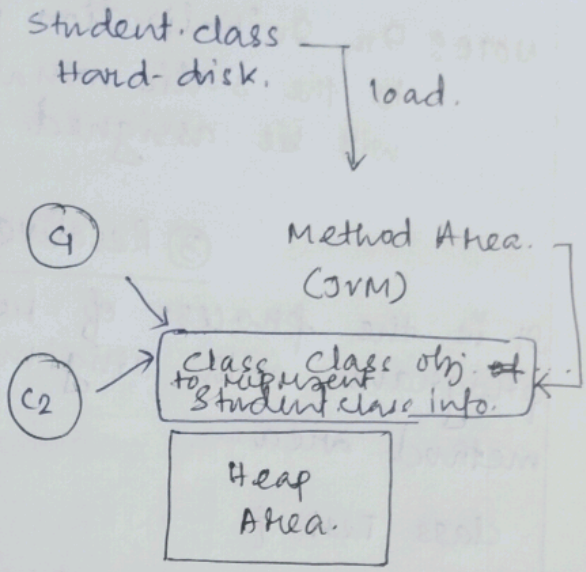
The class Class object can be used by programmer to get class level information like methods' information, variable informations, constructor information etc.

```
import java.lang.reflect.*;
class Student {
    public String getName() {
        return null;
    }
    public int getMarks() {
        return 10;
    }
}
```

```
class Test {
    p s v main (String[] args)
    {
        int count=0;
        Class c = Class.forName
            ~~getDeclaredMe~~("Studen");
        Method[] m = c.
            getDeclared
            Methods();
        for(Method mi : m)
        {
            count++;
            s.o.pln(mi.getName());
        }
        s.o.pln("Total Methods"+
                count);
    }
```

NOTE: For every loaded type only one class object will be created. Even though we are using class multiple times in our program.

```
class Test {
    public static void main(S..[] a)
    {
        Student s1 = new Student();
        class c1 = s1.getClass();
        Student s2 = new Student();
        Class c2 = s2.getClass();
        s.o.pln (c1.hashCode());
        s.o.pln (c2.hashCode());
        s.o.pln (c1 == c2);
    }
}
```



In the above program even though we're using Student Class multiple times only one class Class object got created...

2. Linking

Linking consists of three activities -

1. verify
2. prepare
3. resolution / resolve

(1) verification / verify

It is the process of ensuring that binary representation of a class is structurally correct or not that is JVM will check whether .class file is generated by valid compiler or not. That is whether .class file is properly formatted or not. Internally, Byte code verifier is responsible for this activity. Byte code verifier is the part of "class Loader Sub System". If verification fails then we'll get runtime exception saying "java.lang.verify" error.

② Preparation

In this phase JVM will allocate memory for class level static variables and assign default values.

NOTE: In Initialization phase original values will be assigned to the static variables and here only default values will be assigned.

③ Resolve / Resolution

It is the process of replacing symbolic names in our program with original memory references from method area..

```
class Test {
    p s v main (String[] a) {
        String s = new String("Sushovan");
        Student st = new Student();
    }
}
```

For the above class - class Loader loads Test.class, String.class, Student.class, Object.class.

The names of these classes are stored in constant-pool of Test class.

In resolution phase these names are replaced with original memory level references from method area...

3. Initialization

In this all static variables are assigned with original values and static blocks will be executed from parent to child and from top to bottom...

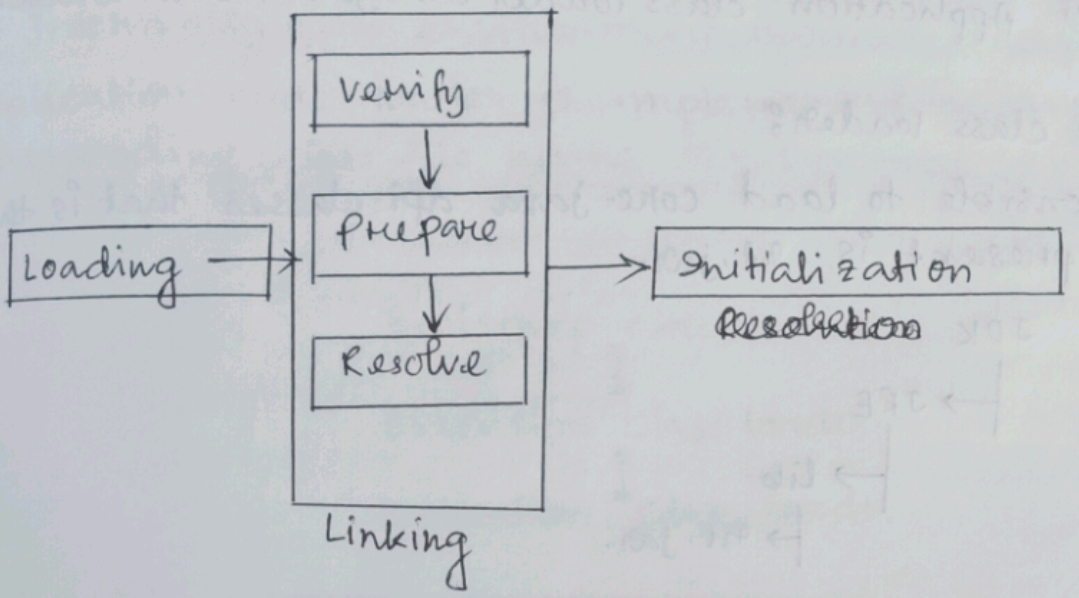


fig: class loading system

"class loader Sub-System is responsible for those processes".

NOTE: while loading, linking and initialization if any error occurs then we'll get runtime exception saying: java.lang.LinkageError

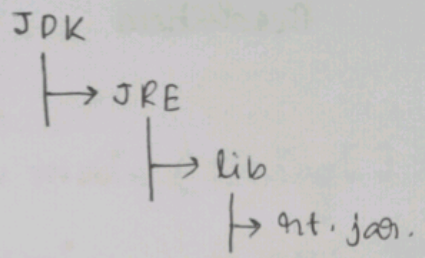
Types of class Loaders

Class loader Sub-Systems contain the following 3 types of class loader -

1. Bootstrap class-loader or primordial class loader
2. Extension class-loader.
3. Application class loader or System class loader..

① Bootstrap class loader:

It is responsible to load core-java api classes that is the classes ~~the~~ present is rt.jar.

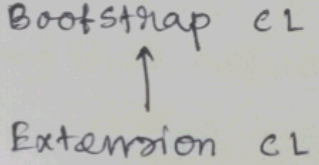


This location is called Bootstrap class path. Bootstrap class loader is responsible to load classes from Bootstrap class path.

Bootstrap class loader is by default available with every JVM it is implemented in native languages - C/C++ not implemented in Java.

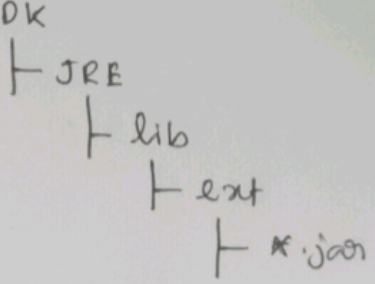
② Extension class loader:

Extension class loader is the child class of Bootstrap class loader.



Extension CL is responsible to load classes from extension class path.

JDK\JRE\lib\ext*.jar



Extension CL is implemented in Java and the corresponding .class file is: sun.misc.Launcher$ExtClassLoader.class

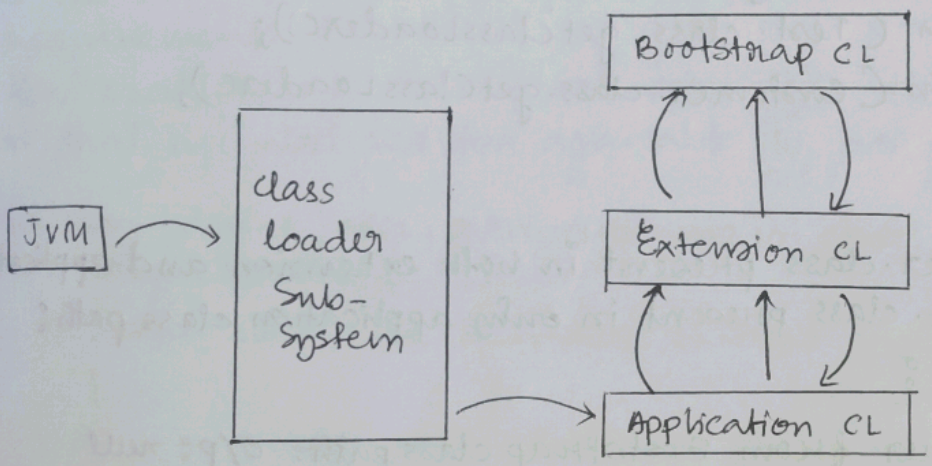
③ Application CL / System CL

Application CL is the child class of Extension CL, this CL is responsible to load classes from Application Class path. It internally uses environment variable classpath.

Application class loader is implemented in java and the corresponding .class file name is - sun.misc.Launcher$AppClassLoader.class

Bootstrap class loader
↕
Extension class loader
↕
Application class loader

How Class loader Works



Class loader follows deligation hirerchy principle. Whenever JVM come across a particular class first it will check whether whether the corresponding .class file is already loaded or not. If it is already loaded in method area then JVM will consider that loaded class.

If it is not loaded then JVM requests class loader Sub-system to load that particular class.

Then, class loader Sub-system hands over the request to application class loader. Application class loader deligates the request to Extension class loader which interned deligates the request to

Bootstrap class loader.

Then, Bootstrap class loader will search in bootstrap class path if it is available then the corresponding class will be loaded by Bootstrap class loader if not available then Bootstrap CL deligates the request to extension CL.

Extension CL will search in Extension class path. If it is available then it will be loaded otherwise extension CL deligates the request to Application CL.

Application class loader will search in Application class-path. If it is available then it will be loaded otherwise we'll get runtime exception saying: NoClassDefFound error

or

classNotFoundException.

```
class Test {
    public static void main (String[ ] a) {
        s.o.pln (String.class.getClassLoader());
        s.o.pln (Test.class.getClassLoader());
        s.o.pln (Customer.class.getClassLoader());
    }
}
```

// Assume Customer.class present in both extension and application class-path and Test.class present in only application class path:

For String.class:

Bootstrap class loader from Bootstrap class path. O/P: null

For Test.class:

Application class loader from Application class path.
O/P: sun.misc.Launcher $ AppClassLoader@ 1919a78

For Customer.class

Extension class loader from Extension class path
O/P: sun.misc.Launcher $ ExtClassLoader@ 1705a81

NOTE: Bootstrap class loader is not java object hence we got null in the first case. But, Extension and Application class-loaders are java objects hence we're getting corresponding output for the remaining two sops... (className @ hashCode in hexadecimal form...)

class loader Sub-System will give the highest priority for Bootstrap Classpath. and then Extension class path followed by Application classpath...

Need of Customizer Class loader.

Default Class loader will load .class file only once even though we are using multiple times .class in our program.
After loading .class file if it is modified outside then default class loader won't load updated version of Class file. (Because .class file already available in method area.
We can resolve this problem by defining our own customizer class loader, the main advantage of customizer class loader is we can control class loading mechanism based on our requirement.
For example we can load .class file seperately everytime so that updated version available to our program.

We can define our own customizer class loader by extending java.lang.ClassLoader class -

```
public class CustClassLoader extends CLassLoader
{
        public class loadClass (String name) throws CLassNot
                                                FoundExcep.
        {
            check for updates and load updated .class
            file and returned corresponding Class
        }
}
class Test {
    P S V main (String[ ] a)
    {   Dog d1 = new Dog ();
        CustClassLoader cl = new CustClassLoader ();
            :
} }     cl.loadClass ("Dog");
```

While developing web-servers and application servers usually we can go customized class loaders to customize class loading mechanism.

What is the need of ClassLoader Class?
We can use java.lang.ClassLoader class to define to define our own customizer class loaders. Every class loader in java should be child class of java.lang.ClassLoader class. Either directly or indirectly hence this class access base class for all customized class-loaders.

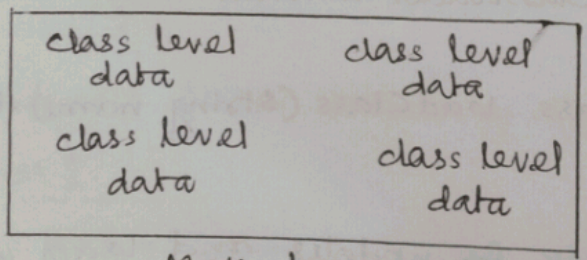
Various Memory Areas Present inside JVM.

Whenever JVM loads and runs a java program it needs memory to store several things like- bytecode, objects, variables etc.
Total JVM memory organized into the following five categories-

① Method Area ③ Stack Memory ⑤ Native Method stacks
② Heap Area ④ PC-Registers

① Method-Area

a. For every JVM one method area will be available.
b. Method area will be created at the time of JVM start-up.
c. Inside method area class level binary data including static variable will be stored.
d. Constant pools of a class will be stored inside method area.

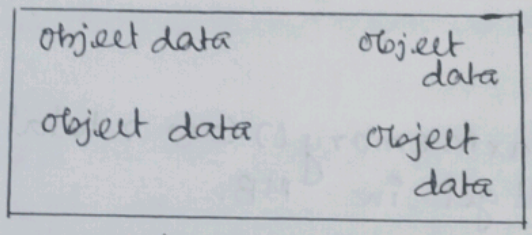


Method Area

e. Method area can be accessed by multiple threads simultaneously.

② Heap-Area:

a. For every JVM one Heap-Area is available.
b. Heap-Area will be created at the time of JVM start-up.
c. objects and corresponding instance variables will be stored in the heap area.
d. Every Array in Java is object only hence arrays also will be stored in the heap-area.
e. Heap-Area can be accessed by multiple threads and hence the data stored in the heap-memory is not thread safe.
f. Heap-Area need not be continuous.



Heap-Area.

Program to display heap memory statistics:

A java application can communicate with JVM by using runtime object.

Runtime class present in java.lang package and it is a singleton class.

We can create Runtime object as follows:

```
Runtime r = Runtime.getRuntime();
```

Once, we got Runtime object we can call the following methods on that object:

1) maxMemory():
It returns number of bytes of maxMemory allocated to the heap.

2) totalMemory():
It returns number of bytes of total memory allocated to the heap. (initial memory).

3) freeMemory():
It returns number of bytes of free memory present in the heap.

```
class HeapDemo {
    public static void main (String[ ] a) {
        double mb = 1024 * 1024;
        Runtime r = Runtime.getRuntime ();
        System.out.println ("Max-memory" + r.maxMemory());
        System.out.println ("Total-memory" + r.totalMemory());
        System.out.println ("Free-memory" + r.freeMemory());
        System.out.println ("Consumed-memory" +
                r.totalMemory() - r.freeMemory());
    }
}
```

* If we divide → r.maxMemory() (or) r.totalMemory() --- etc. with (1024*1024), we'll get in MB.

r.maxMemory() (or) r.totalMemory() - gives result in number of bytes...

How to set maximum and minimum heap sizes:

Heap memory in finite memory but based on our requirement we can set maximum and minimum heap sizes that is we can increase or decrease the heap size based on our requirement.

We can use the following flags with java command-

i) -Xmx : to set maximum heap size. (maxMemory())

```
java -Xmx512m HeapDemo
```

□ This command will set maximum heap size as 512 mb.

ii) -Xms : to set minimum heap size.

```
java -Xms64m HeapDemo
        ClassName.
```

□ This command will set minimum heap size as 64 mb (totalMemory() will be affected).

```
java -Xmx512m -Xms64m HeapDemo
```

```
Max-M: 494.4
Total-M: 61.87
Free-M: 61.2374
Consumed-M: 0.6821...
```

③ Stack - Memory:

a) For every thread JVM will create a seperate stack at the time of Thread creation.

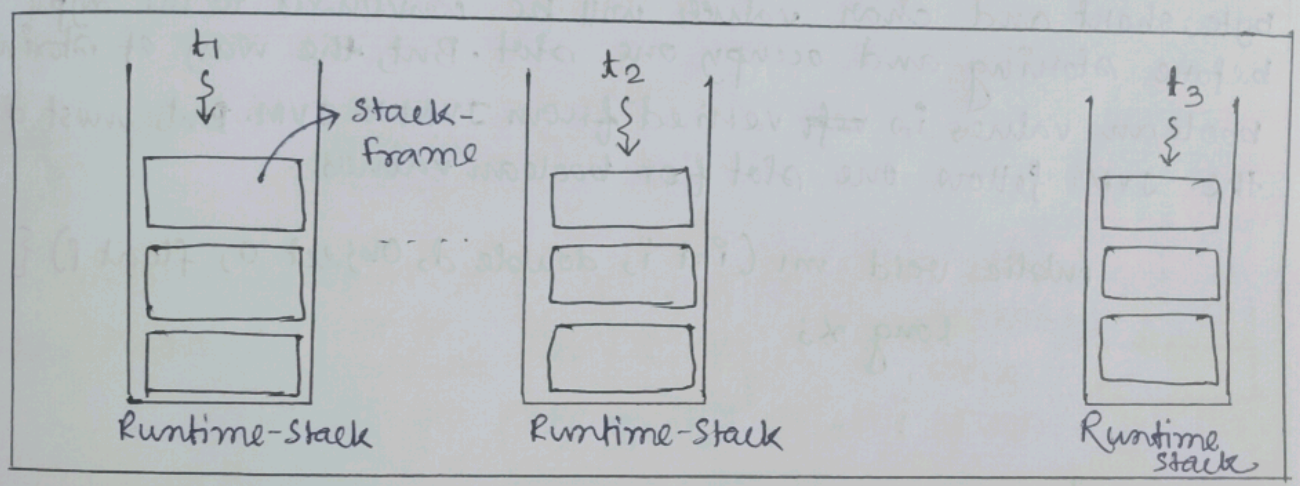
b) Each and every method call performed by that thread will be stored in the stack. Including local variables also.

c) After completing a method the corresponding entry from the stack will be removed.

d) After completing all method calls the stack will become empty and that empty stack will be destroyed by the JVM just before terminating the thread.

f) Each entry in the stack is called stack - frame or activation record...

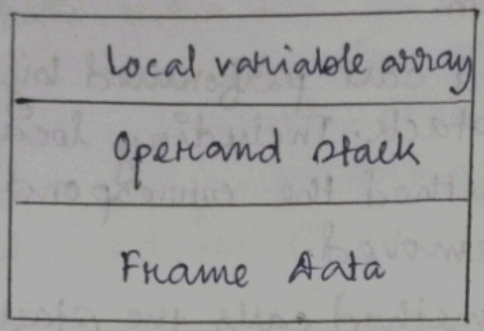
g) The data stored in the stack is available only for the corresponding thread and not available to the remaining threads. Hence this data is thread - safe.



Stack - Memory

Stack frame Structure

Each stack frame contains 3 parts.

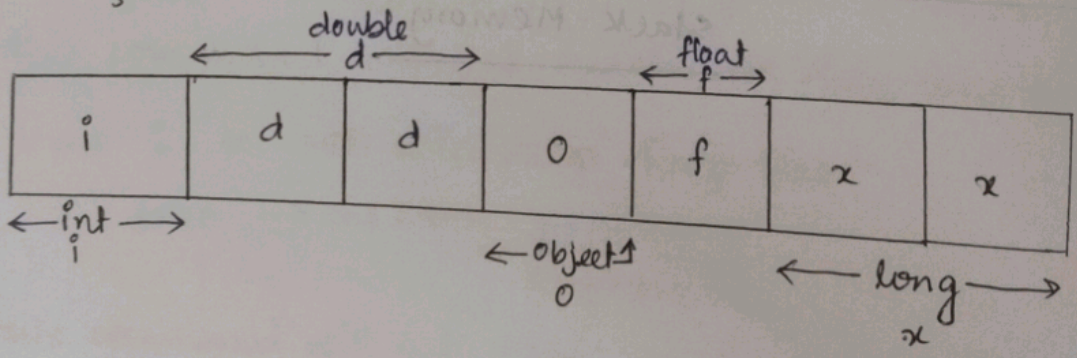


Stack-Frame.

i) local variable array:

It contains all parameters and local variables of the method.
Each slot in the array is of 4 bytes.
Values of type int, float and reference occupy 1 entry in the array.
Values of double and long occupy two consequtive entries in the array.
byte, short and char values will be converted to int type before storing and occupy one slot. But, the way of storing boolean values is ~~not~~ varied from JVM to JVM. But, most of the JVM follow one slot for boolean values.

```
public void m1 (int i, double d, Object o, float f) {
    long x;
    .
    .
    .
}
```



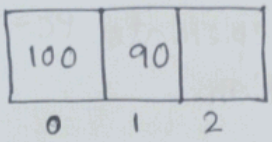
ii) Operand stack:

JVM uses operand stack as work-space.
some instructions can push the values to the operand stack and some instructions can pop values from operand stack and some instructions can perform required operations.

Program
iload-0
iload-1
iadd
istore-2

Before-starting

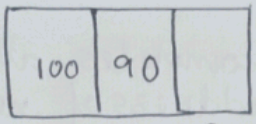
Local variable array



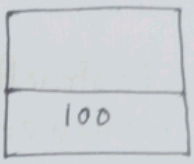
operand-stack

After iload-0

Local-variable array

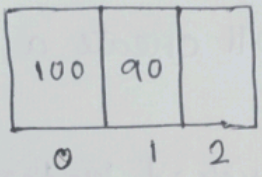


Operand-stack

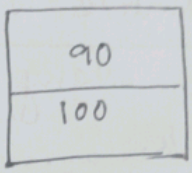


After iload-1

Local-variable array

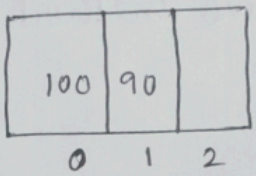


Operand-Stack

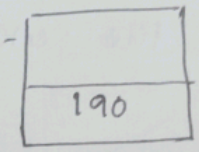


After iadd

Local-variable array

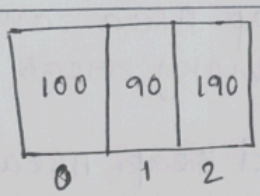


Operand-stack

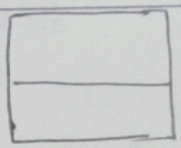


After istore-2

Local-variable array



Operand-stack



iii) Frame Data :

Frame Data contains all symbolic references related to that method it also contains a reference to exception table, which provides corresponding catch-block info. in case of exception.

④ PC-Registers : (Program Counter Registers)

For every thread a ~~spe~~ seperate PC-register will be created at the time of thread creation.

PC-Register contains the address of current executing instruction.

Once instruction execution completes automatically PC-Register will be incremented to hold address of next information.

⑤ Native-Method stack :

For every thread JVM will create a seperate native method stack.

All native method calls invoked by the thread will be stored in the corresponding native method stack

NOTE :

a. Method Area, Heap-Area and Stack area are considered as important memory areas with respect to programmer.

b. Method-Area and Heap-Area are for JVM.

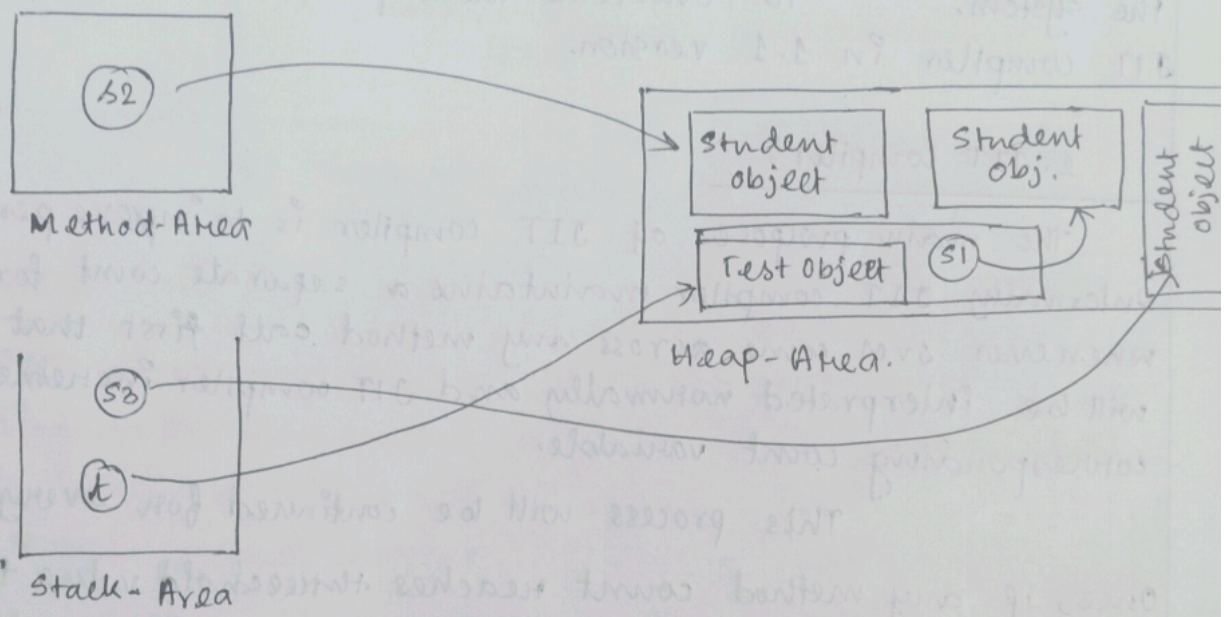
c. Stack-Area, PC-Registers and Native-Method-stack are per thread.

For every JVM – One Heap Area
One Method Area

For every Thread – One stack Area
One PC Reg.
One Native-Method stack.

d. Static variables will be stored in Method-Area.
e. Instance variables will be stored in Heap-Area.
f. Local variables will be stored in Stack-Area.

```
class Test {
    Student s1 = new Student ();
    static Student s2 = new Student ();
    public static void main (String[] a) {
        Test t = new Test ();
        Student s3 = new Student ();
    }
}
```



⑥ Execution Engine:

a. This is the central component of JVM.
b. Execution engine is responsible to execute java class files.
c. Execution engine mainly contains two components
    1. Interpreter
    2. JIT Compiler.

1. Interpreter:

It is responsible to read bytecode and interpret into machine code (Native code) and execute that machine code line by line.

The problem with interpreter is it interprets everytime even same method invoked multiple times which reduces performance of the system. To overcome these problems Sun people introduced JIT compiler in 1.1 version.

2. JIT Compiler:

The main purpose of JIT compiler is to improve performance. Internally JIT compiler maintains a seperate count for every method whenever JVM come across any method call first that method will be interpreted normally and JIT compiler increments corresponding count variable.

This process will be continued for every method. Once, if any method count reaches threeshold value then JIT compiler identifies that, method is a repeatedly used method. (Hotspot). Immediately, JIT compiler compiles that method and generates the corresponding native code. Next time, JVM come across that method call then JVM usage native code directly and executes it instead of interpreting once again. So that, performance of the system will be improved.

The threeshold count varied from JVM to JVM.

Some, advanced JIT compilers will recompile generated native code if count reaches threeshold value second time. So that, more optimised machine code will be generated.
Internally, Profiler, which is the part of JIT compiler is responsible to identify Hotspots.

NOTE:

1. JVM interprets total program at least once.
2. JIT compilation is applicable only for repeatedly required methods not for every method.

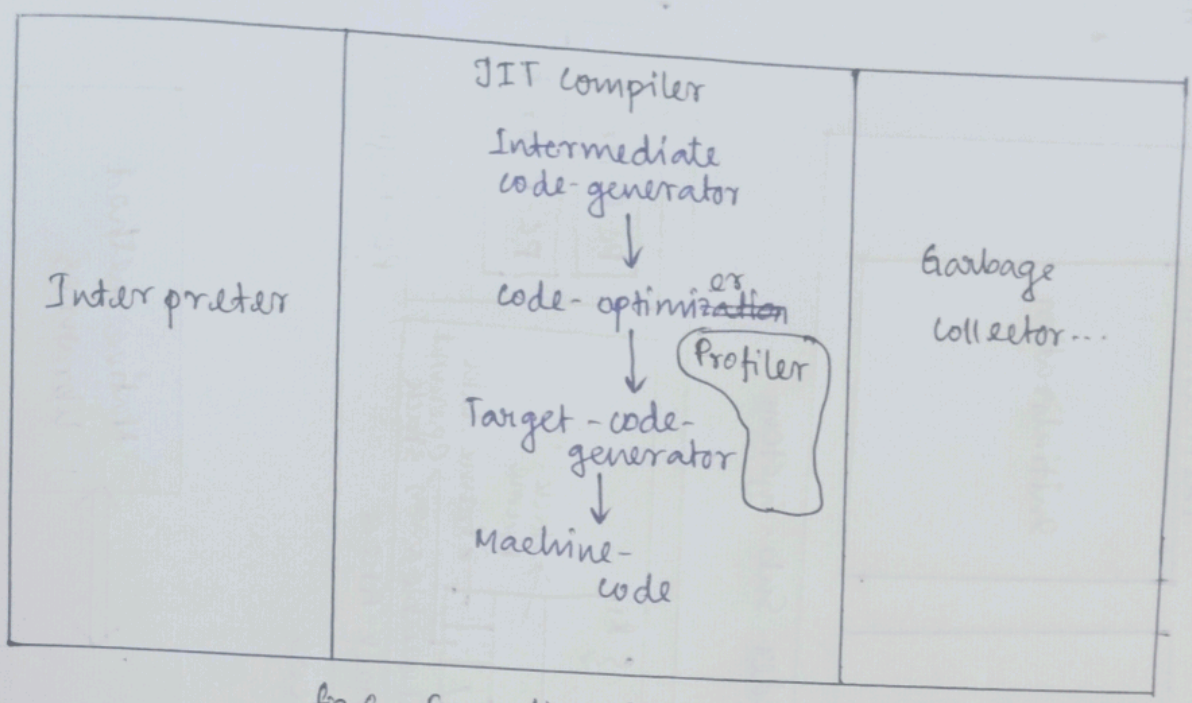
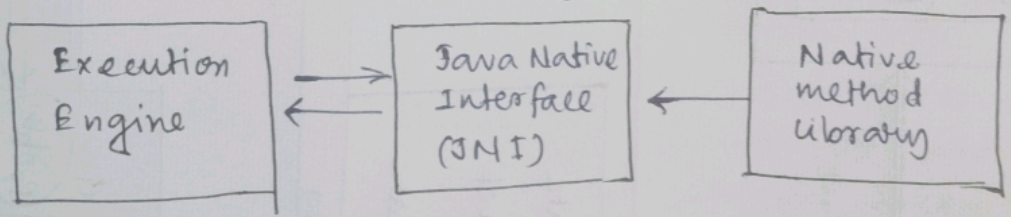


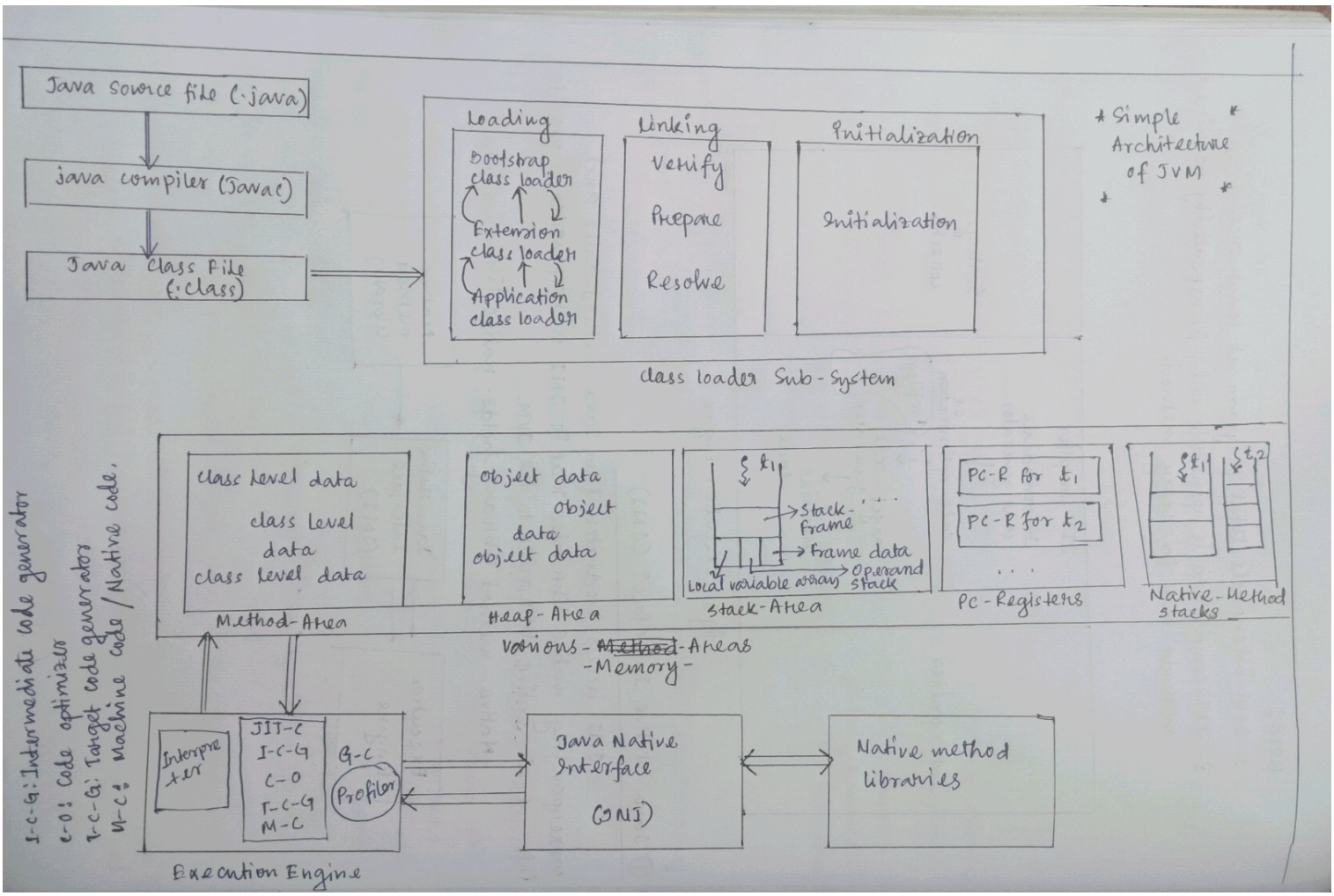
fig: Execution-Engine

⑦ Java Native Interface: (JNI)

JNI access mediator for Java method calls and corresponding native libraries. that is JNI is responsible to provide info. about native libraries to the JVM.

Native method library holds- native libraries info.




Java source file (.java)
java compiler (Javac)
Java Class File (.class)
Loading
Bootstrap class loader
Extension class loader
Application class loader
Linking
Verify
Prepare
Resolve
Initialization
Initialization
class loader Sub-System
* Simple Architecture of JVM *
class level data
class level data
class level data
Method-Area
object data
object data
object data
Heap-Area
t_1
stack-Frame
frame data
Local variable array
Operand stack
Stack-Area
PC-R for t_1
PC-R for t_2
PC-Registers
t_1
t_2
Native-Method stacks
various-Method-Areas
-Memory-
Interpreter
JIT-C
I-C-G
C-O
T-C-G
M-C
G-C
Profiler
Execution Engine
Java Native Interface
(JNI)
Native method libraries
I-C-G: Intermediate code generator
C-O: Code optimizer
T-C-G: Target code generator
M-C: Machine code / Native code.

Class-file Structure

```
classFile {
    magic_number;
    minor_version;
    major_version;
    constant_pool_count;
    constant_pool[ ];
    access_flags;
    this_class;
    super_class;
    interface_count;
    interface[ ];
    fields_count;
    fields [ ];
    methods_count;
    methods[ ]
    attributes_count;
    attributes[ ];
}
```

i) magic_number:

The first four bytes of class file is magic-number. this is a pre-defined value used by JVM to identify class file is generated by valid compiler or not. The value should be 0XCAFBABE

NOTE: Whenever we're executing a java class if JVM unable to find a valid magic_number then we'll get runtime exception saying: java.lang.ClassFormatError: Incompatible magic value.

ii) minor_version and major_version:

Major and minor version represents class file version. JVM will use these versions to identify which version of compiler generates the current class file...

M.m

Major version — minor version

1.5v — 49.0
1.6v — 50.0
1.7v — 51.0

NOTE: Lower version compiler generated .class file can be run by higher version JVM.
But, higher version compiler generated .class files can't be run by lower version JVMs.
If we're trying to run we'll get runtime exception saying: UnsupportedClassVersionError

javac 1.6V java 1.7V ✓
javac 1.7V java 1.6V. ✗
↳ RE: UnsupportedClassVersionError.

iii) Constant-Pool Count:

It represents number of constant presents in constant pool

iv) Constant-Pool array[]:

It represents information about constants present in constant-pool.

v) access-flags:

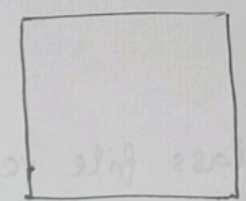
It provides information about modifiers which are declared to the class.

vi) this_class:

It represents fully qualified name of the class.

vii) super_class:

It represents fully qualified name of immediate super class of current class.


Test.class

this_class: Test
super_class: Object

viii) interface_count:

It returns number of interfaces implemented by current class.

ix) interface[]:

It returns interfaces information implemented by current class.

x) fields count:

It represents number of fields present in current class.

xi) fields[]:

It represents fields' information present in current class.

xii) methods_count:

It represents number of methods present in current class.

xiii) methods[]:

It provides information about all methods present in current class.

xiv) attributes_count:

It returns number of attribute present in current class.

xv) attributes[]:

It provides information about all attributes present in current class.

- javap -verbose Test.class

some info. about the class we can get by this command.